

Intelligent Control

Session 5: Trajectory Tracking, and a First Nonlinear Controller

Dr. Gerardo Flores

SENG 5360 · Texas A&M International University ·

Today

1. Regulation versus tracking — *holding a point, following a path*
2. The tracking error and its dynamics
3. A controller that cancels the nonlinearity — *computed torque*
4. What it can do now, and what it costs
5. Simulation lab

Learning objectives

- write the tracking error and its dynamics for the pendulum;
- build the computed-torque controller and explain each of its four terms;
- compute the torque it asks for along a path, and see when the motor limits it;
- predict the offset left by a wrong model, and know the three fixes.

Before we start: get the code

Every example and exercise of today runs from **one** Python file:

`pendulum_tracking.py`

1. Go to the course repository on GitHub: `github.com/gfloresc/intelligent-control`
2. Open the folder `code/` and download `pendulum_tracking.py`.
3. Open it in Google Colab and run it. You should see the pendulum swing from the bottom to the top, following a dashed "ghost" pendulum.

It works like `pendulum_inverted.py` of Session 4: a list `SCENARIOS` near the top, one entry per example of the slides, and a single line to change,

`SCENARIO = 'swing_up'` $\longrightarrow$ `SCENARIO = 'hold_side'`.

The animation shows the actual pendulum (solid) and the desired one (dashed), and plots the angle, the error and the torque against time.

In `pendulum_tracking.py`, mark `[PATH] ... [MODEL]`: the grey boxes on the slides tell you which scenario to run and which lines to look at. Search the file for the mark.

Where we are

Module 1 gave us a toolbox: linearize at an equilibrium, place the eigenvalues, test on the nonlinear model. It balanced the pendulum upright, even with a wrong mass, a push and a noisy sensor.

It also showed its limit. Everything was built *near one point*:

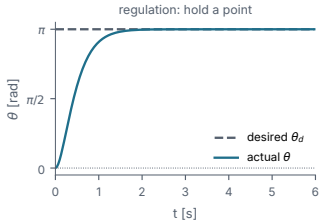
- Session 3: the linear model is only valid in a neighbourhood, and the gain only works there; with a 1.2 N m motor, a large initial tilt already made the pendulum fall;
- Module 1 never brought the pendulum *up* from the bottom. That trip passes through angles where the linearization is simply wrong.

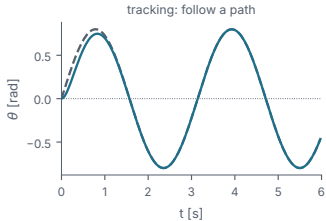
Module 2 removes that limit. Instead of approximating the nonlinear model, we use it *as it is*, inside the controller. Today we build the first such controller for the pendulum and watch it do what Module 1 could not: swing the pendulum up along a chosen path.

Along the way we meet the two questions that organise the whole module:

- what exactly do we want the system to do (**regulation** or **tracking**), and
- how do we know it will do it, once the equations are no longer linear (**Lyapunov**, next session).

Regulation versus tracking





Regulation: drive the state to a *constant* target and hold it there. The desired angle is a number, $\theta_d = \pi$. All of Module 1 was regulation. Examples: hold the pendulum up, keep a room at 22 °C, hover a drone.

Tracking: make the state *follow* a target that moves in time. The desired angle is a function $\theta_d(t)$, the **reference trajectory** (we will also call it the path), with its own $\dot{\theta}_d(t)$ and $\ddot{\theta}_d(t)$. Examples: a robot arm tracing a weld, a drone on a route, a car keeping its lane.

Regulation is tracking with $\dot{\theta}_d = \ddot{\theta}_d = 0$. From now on we always write the tracking problem, and regulation comes out as a special case.

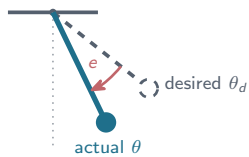
In `pendulum_tracking.py`, mark `[PATH]`: `path(t)` returns $\theta_d, \dot{\theta}_d, \ddot{\theta}_d$. Run `'hold_top'` (regulation) and `'sine'` (tracking).

The tracking error

Everything in control design is about one signal, the **tracking error**:

$$e(t) = \theta(t) - \theta_d(t), \qquad \dot{e} = \dot{\theta} - \dot{\theta}_d, \qquad \ddot{e} = \ddot{\theta} - \ddot{\theta}_d.$$

e is *where we are minus where we should be*. The goal of the controller, in both regulation and tracking, is simply $e(t) \to 0$.



Two remarks that save confusion later:

- The tilt of Session 4, $\delta\theta = \theta - \pi$, is this same error for the constant target $\theta_d = \pi$. Nothing new, only more general.
- $\dot{e}$ is *not* the pendulum velocity. If the pendulum is at rest while the target moves, $\dot{e} = -\dot{\theta}_d \neq 0$: we are falling behind.

The plan of every controller in this module: write the differential equation that e obeys, then choose u so that this equation sends e to zero. The plant is given; the error equation is *ours* to shape.

In `pendulum_tracking.py`, mark `[ERR]`: two lines of `controller()`: `e = th - thd` and `e_dot = om - thd_dot`.

Step 1: the equation the error obeys

Start from the pendulum of Session 1, with the real mass and the motor torque u:

$$m\ell^2\,\ddot{\theta} = -mg\ell\sin\theta - b\,\dot{\theta} + u.$$

We want an equation in which the unknown is e, not θ. The path $\theta_d(t)$ is a known function of time, so we write the pendulum in terms of the error:

$$\theta = e + \theta_d, \qquad \dot{\theta} = \dot{e} + \dot{\theta}_d, \qquad \ddot{\theta} = \ddot{e} + \ddot{\theta}_d.$$

Substituting the three in the pendulum equation and moving $m\ell^2\ddot{\theta}_d$ to the right:

$$\boxed{m\ell^2\,\ddot{e} = -mg\ell\sin(e + \theta_d) - b\,(\dot{e} + \dot{\theta}_d) - m\ell^2\,\ddot{\theta}_d + u}$$

This is the **error dynamics**: its state is $(e, \dot{e})$; everything else is the input u or a known signal of the path. The right-hand side is four torques acting on the error:

- $-mg\ell\sin(e + \theta_d)$: gravity, which depends on where the pendulum is, the path plus the error;
- $-b\,(\dot{e} + \dot{\theta}_d)$: friction;
- $-m\ell^2\,\ddot{\theta}_d$: the torque the *path* demands to accelerate the pendulum along it (zero for regulation);
- u: ours.

Because $\theta_d(t)$ sits inside the sine, the error dynamics of a tracking problem is *time-varying*, even though the pendulum itself is not.

Exercise 1: the cheapest swing

The pendulum must swing around the bottom along $\theta_d(t) = A\sin\omega t$, with a small amplitude $A = 0.2$ rad.

1. Find the torque that keeps $e \equiv 0$. Using $\sin(A\sin\omega t) \approx A\sin\omega t$, show that it is $u = A\big(mg\ell - m\ell^2\omega^2\big)\sin\omega t + bA\omega\cos\omega t$.
2. At what frequency does the gravity-inertia term cancel? What is that frequency, physically?
3. Now swing around the *top*. Before computing: does gravity pull the pendulum back toward the center, or push it away? Use that to predict whether a cheap frequency exists, then check it.

Hint. $e \equiv 0$ means $\dot{e} = \ddot{e} = 0$ too. Around the top, $\sin(\pi + x) = -\sin x$.

In `pendulum_tracking.py`, mark `[PATH]`: run `'swing_slow'` ($\omega = 2$), `'swing_natural'`, `'swing_fast10'` ($\omega = 10$) and `'swing_top'`. They start on the path, so the printed peak torque is the torque of the path itself.

Exercise 1: solution

Step 1. With $e = \dot{e} = \ddot{e} = 0$ the boxed equation gives $u = m\ell^2\ddot{\theta}_d + b\dot{\theta}_d + mg\ell\sin\theta_d$. Substituting $\dot{\theta}_d = A\omega\cos\omega t$, $\ddot{\theta}_d = -A\omega^2\sin\omega t$ gives the formula. Keep this torque: the controller of the next slides is built from it.

Step 2. The sin term is a fight between gravity ($mg\ell$) and inertia ($m\ell^2\omega^2$). It vanishes at $\omega = \sqrt{g/\ell} = 5.72\,\text{rad/s}$, the **natural frequency** of the hanging pendulum (Session 3). There gravity alone produces the acceleration the path needs, and the motor only pays for friction: $0.057\,\text{N m}$, against $0.26\,\text{N m}$ at $\omega = 2$ and $0.62\,\text{N m}$ at $\omega = 10$. It is pushing a swing at its own rhythm. (Because of friction the total torque is smallest slightly below, at $5.66\,\text{rad/s}$; the difference is negligible.)

Step 3. Around the top the gravity term changes sign: the sin term becomes $-A\big(mg\ell + m\ell^2\omega^2\big)$, and it is never zero. Up there gravity pushes the pendulum away instead of back, so the motor must fight it at every frequency.

Lesson. A good path works *with* the dynamics of the plant, not against it.

Step 2: cancel what you know, then add what you want

Exercise 1 found the torque that keeps $e = 0$ once on the path; the pendulum starts off it, so we need more. *Choose u to cancel the three known torques, and add a fourth that we design.*

Step 2a. Cancel. Put into u the exact opposite of gravity, friction and the path torque:

$$u = mg\ell \sin(e + \theta_d) + b\,(\dot{e} + \dot{\theta}_d) + m\ell^2\,\ddot{\theta}_d + (\dots).$$

Substituting in the boxed equation, those three pairs cancel and only $m\ell^2\ddot{e} = (\dots)$ remains.

Step 2b. Add. Now the error behaves like a free mass, $m\ell^2\ddot{e} = (\dots)$, and we know from Session 2 how to make a free mass come to rest: a spring and a damper on the error,

$$(\dots) = m\ell^2\,\big(-k_p\,e - k_d\,\dot{e}\big).$$

The next slide collects the result.

Step 2 (continued): the controller and its error equation

Step 2c. Collect. The controller and the error equation it produces are

$$\boxed{u = mg\ell \sin(e + \theta_d) + b\,(\dot{e} + \dot{\theta}_d) + m\ell^2\,(\ddot{\theta}_d - k_d\,\dot{e} - k_p\,e)}$$

Check it: put this u in the boxed error dynamics of Step 1. Gravity, friction and the path torque cancel term by term, and dividing what is left by $m\ell^2$:

$$\boxed{\ddot{e} + k_d\,\dot{e} + k_p\,e = 0}$$

Since $e + \theta_d = \theta$ and $\dot{e} + \dot{\theta}_d = \dot{\theta}$, in practice the first two terms are computed from the measured angle and velocity: $mg\ell \sin\theta + b\dot{\theta}$. Same numbers.

No approximation was made. The error equation is linear for *every* angle, not only near one point; with $k_p, k_d > 0$ its eigenvalues have negative real part (Session 2), so $e \to 0$ from any start. Larger gains make e die faster, but ask more torque for the same error. Today we use $k_p = 16$, $k_d = 8$.

In `pendulum_tracking.py`, mark `[CT]`: the last line of `controller()`; `v` is the bracket $\ddot{\theta}_d - k_d\dot{e} - k_p e$.

Reading the controller

$$u = \underbrace{mg\ell\sin(e+\theta_d)}_{\text{1. gravity}} + \underbrace{b\,(\dot{e}+\dot{\theta}_d)}_{\text{2. friction}} + \underbrace{m\ell^2\,\ddot{\theta}_d}_{\text{3. feedforward}} + \underbrace{m\ell^2\,(-k_d\,\dot{e} - k_p\,e)}_{\text{4. feedback}}$$

1. **Gravity compensation.** Whatever gravity does to the pendulum, the motor undoes. Held horizontal, at $\theta = \pi/2$, it supplies exactly $mg\ell = 1.47\,\text{N m}$; at the top or the bottom it supplies nothing, because there gravity has no lever arm.
2. **Friction compensation.** Same idea, for the friction term.
3. **Feedforward.** The torque needed to accelerate along the path, applied *before* any error appears. For regulation it is zero.
4. **Feedback.** The PD of Session 3, now acting on the *error* and scaled by the inertia $m\ell^2$. This is the only term that reacts to a mistake.

Terms 1 to 3 are the model, computed from the measured $\theta = e + \theta_d$, $\dot{\theta}$ and the path; term 4 is the correction. Hence the name **computed torque**. In robotics books it is also called **feedback linearization**: the feedback made the error equation linear.

What it needs

The model (m, ℓ, b), the full state (θ, $\dot{\theta}$) and the path with its two derivatives. Each is a possible weak point; we test the first one later in this session.

Session 4 versus today

Both controllers have a PD. The difference is what they do about gravity.

	Linear design (Sessions 3, 4)	Computed torque (today)
gravity	approximated by $+mg\ell\,\delta\theta$ near the top	cancelled exactly, $mg\ell\sin(e+\theta_d)$
where it works	a neighbourhood of one equilibrium	every angle
condition	the gain must *beat* gravity, $k_1 > mg\ell$	the motor must supply what the controller cancels along the path
needs the model?	only to compute $\mathbf{K}$ once	at every instant, inside u
error equation	$m\ell^2\delta\ddot{\theta} + (b+k_2)\delta\dot{\theta} + (k_1 - mg\ell)\delta\theta = 0$	$\ddot{e} + k_d\dot{e} + k_p e = 0$

Look at the last row. In Session 4 the "spring" was $k_1 - mg\ell$, and the mass could weaken it or make it negative. Today the spring is k_p, chosen by us, and gravity is simply not there. That is what buying the model gives us.

The price is in the fourth row: the controller must know m, ℓ and b at every instant. A wrong value means an imperfect cancellation, and we will see exactly what that costs.

Exercise 2: is friction compensation worth it?

Remove the friction term from the computed torque, keeping $k_p = 16$, $k_d = 8$:

$$u = mg\ell \sin(e + \theta_d) + m\ell^2 \left(\ddot{\theta}_d - k_d \dot{e} - k_p e\right).$$

1. Write the error equation this u produces.
2. Compare two tasks: holding a constant angle, and spinning the pendulum forever at constant speed, $\theta_d = \Omega t$ with $\Omega = 2\,\mathrm{rad/s}$. In which one does an error remain, and how large is it?

Hint. Once the motion is steady, $\dot{e} = \ddot{e} = 0$.

In `pendulum_tracking.py`, mark `[CT]`: run `'spin'` and `'spin_no_fc'`; the second sets `fc=False`, which removes the friction term.

Exercise 2: solution

Step 1. Gravity and the path torque still cancel, friction does not. Dividing by $m\ell^2$:

$$\ddot{e} + \left(k_d + \frac{b}{m\ell^2}\right)\dot{e} + k_p\, e = -\frac{b}{m\ell^2}\,\dot{\theta}_d, \qquad \frac{b}{m\ell^2} = 1.11.$$

Step 2. Holding an angle, $\dot{\theta}_d = 0$: the right side vanishes and the damping only grows from 8 to 9.11. Still positive, so $e \to 0$: skipping the compensation is harmless.

Spinning, $\dot{\theta}_d = \Omega$ stays on the right side. At steady state

$$e_{ss} = -\frac{b\,\Omega}{m\ell^2 k_p} = -\frac{0.05 \times 2}{0.045 \times 16} = -0.139\,\text{rad},$$

the pendulum spins forever 8° behind the path (simulation: $-0.1386\,\text{rad}$). It does pass the top every turn: gravity is still cancelled at every angle.

Lesson. Model terms that multiply $\dot{\theta}_d$ or $\ddot{\theta}_d$ vanish when the target stands still. Tracking needs a better model than regulation.

Where we stand: the computed-torque controller

Everything so far fits on one slide. Come back to it whenever you get lost.

1. **The goal.** Make the tracking error $e = \theta - \theta_d$ go to zero.
2. **The error dynamics** (Step 1): $m\ell^2\ddot{e} = -mg\ell\sin(e + \theta_d) - b(\dot{e} + \dot{\theta}_d) - m\ell^2\ddot{\theta}_d + u$.
3. **The controller** (Step 2): cancel the three known torques, add a PD on the error,

$$u = mg\ell\sin(e + \theta_d) + b(\dot{e} + \dot{\theta}_d) + m\ell^2\big(\ddot{\theta}_d - k_d\dot{e} - k_p e\big).$$

4. **The result:** $\ddot{e} + k_d\dot{e} + k_p e = 0$ at every angle, so $e \to 0$ for any $k_p, k_d > 0$. We use $k_p = 16$, $k_d = 8$.
5. **The price:** it needs the model, the full state, a smooth path, and a motor strong enough to supply what it cancels.

The rest of the session tests the claim of item 4 on a hard task, and then the price of item 5.

Question: can we swing it up now?

Module 1 only balanced the pendulum near the top; it never brought it up from the bottom. Let us try it as a *tracking* problem: design a path from 0 to π and follow it.

The path. We want it to start and end at rest, with no jumps in velocity. A half cosine does that:

$$\theta_d(t) = \frac{\pi}{2}\left(1 - \cos\frac{\pi t}{T}\right), \qquad 0 \le t \le T, \qquad \text{then } \theta_d = \pi.$$

Its derivatives, which the controller needs:

$$\dot\theta_d = \frac{\pi}{2}\,\frac{\pi}{T}\,\sin\frac{\pi t}{T}, \qquad \ddot\theta_d = \frac{\pi}{2}\left(\frac{\pi}{T}\right)^2 \cos\frac{\pi t}{T}.$$

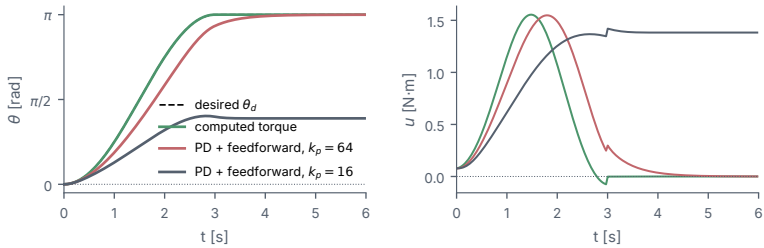
At $t = 0$ and $t = T$ the velocity is zero; the maximum velocity, $\frac{\pi}{2}\frac{\pi}{T}$, is reached halfway, when the pendulum is horizontal. We take $T = 3\,\text{s}$: maximum velocity $1.64\,\text{rad/s}$.

The test. Pendulum hanging at rest, computed torque with $k_p = 16$, $k_d = 8$, motor limit $3\,\text{N m}$. For comparison we also run a *PD with acceleration feedforward*: the same bracket, but without gravity and friction compensation (we call it "PD" for short):

$$u_{\text{PD}} = m\ell^2\left(\ddot\theta_d - k_d\dot e - k_p e\right).$$

In `pendulum_tracking.py`, mark `[PATH]`: the `'swing'` branch of `path()`. Run `'swing_up'`, then `'swing_up_pd'` and `'swing_up_pd_high'`.

What happens



- **Computed torque** (green): the actual angle sits on top of the dashed path from the start, because the pendulum begins where the path begins (at rest at 0); the error never exceeds 0.001 rad. The torque peaks at 1.55 N m when the pendulum is horizontal, then drops to zero at the top, where gravity has no lever arm.
- **PD + feedforward,** $k_p = 16$ (grey): stuck at 1.22 rad. Its spring is $m\ell^2 k_p = 0.72$ N m/rad, below $mg\ell = 1.47$: the Session 4 rule says it cannot even hold the pendulum, let alone raise it.
- **PD + feedforward,** $k_p = 64$**,** $k_d = 16$ (rose): the spring is now 2.88 N m/rad, so it gets up, but lagging up to 0.52 rad behind the path on the way.

The PD fights gravity with a spring; the computed torque removed gravity from the problem.

Exercise 3: why not a straight line?

The simplest path from 0 to π in T seconds is a straight line, $\theta_d = \pi t/T$, then $\theta_d = \pi$. Why did we use the half cosine instead?

1. Sketch θ_d, $\dot{\theta}_d$ and $\ddot{\theta}_d$ for the line. What happens at $t = 0$ and $t = T$?
2. What torque would keep $e \equiv 0$ at those two instants? What does the real pendulum do instead?
3. What property must a path have so that it can be followed? Does the half cosine have it?

In `pendulum_tracking.py`, mark `[PATH]`: the `'ramp'` branch of `path()`. Run `'ramp_up'` and compare its error with `'swing_up'`.

Exercise 3: solution

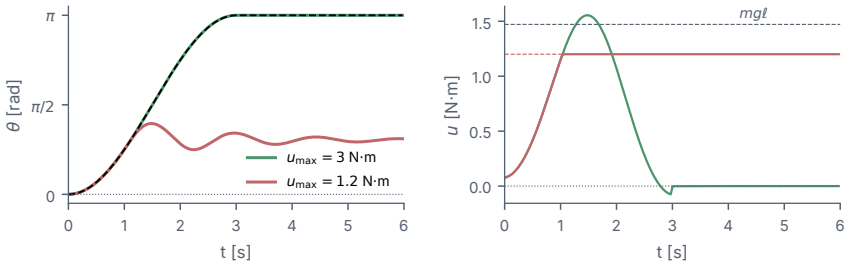
Step 1. The velocity $\dot{\theta}_d$ is 0, then jumps to π/T, then jumps back to 0. An instant change of speed means an infinite acceleration: mathematically an impulse. A real motor can only approximate it with a very large but finite acceleration, during a very short time.

Step 2. Keeping $e \equiv 0$ needs $u = m\ell^2\ddot{\theta}_d + \ldots$: an *infinite* torque at $t = 0$ and $t = T$. No motor has it, so the pendulum cannot jump to speed: it falls behind at the start (-0.096 rad) and overshoots the top (0.10 rad), a hundred times the error of the half cosine (0.001 rad).

Step 3. The velocity of the path must be continuous, with no jumps, and its acceleration $\ddot{\theta}_d$ must stay bounded, so that the torque stays finite. And it must start and end with the velocity the pendulum really has: zero. The half cosine has a continuous velocity, zero at both ends, and a bounded acceleration; its acceleration still jumps by a finite amount at the two ends, which is the kick and the brake of the torque.

Lesson. The path is part of the design. A controller can only follow a path that the motor can produce, and choosing it well is often worth more than tuning gains.

The motor limit



With $u_{max} = 1.2\,\text{N m}$ the torque hits the limit before the horizontal and stays there. The pendulum cannot pass and falls back. At $t = 6\,\text{s}$ it is still settling (0.974 rad); a longer run shows it comes to rest at 0.954 rad, where $mg\ell \sin\theta = u_{max}$: $\theta = \arcsin(1.2/1.4715)$. It is exactly the "lower saturated equilibrium" of Session 3. With the motor stuck at its limit, the controller no longer matters, only the limit does.

In `pendulum_tracking.py`, mark `[CT]`: run `'swing_limited'`; compare with `'swing_fast'` ($T = 1\,\text{s}$, limit $3\,\text{N m}$), which works and peaks at $1.78\,\text{N m}$.

The motor limit: the lesson

No controller beats a weak motor

The computed torque cancels gravity *only if the motor can supply it*. To *hold* the pendulum horizontal, $u_{\max} \geq mg\ell$ is necessary; a strict inequality leaves margin for the feedback. Passing through the horizontal on our path asks slightly more: there $\ddot{\theta}_d = 0$, so $u = mg\ell + b\dot{\theta}_d$. A faster path does not lower this number (T enters only through the friction term) and raises the kick and the brake. None of this is needed to reach the top by other means (see below).

Compare the two parts of the controller: the model terms of u are *required* torque, set by physics and the path; the feedback term is *extra* torque, set by our gains. A motor must cover the first with margin for the second. Check this before choosing the path and the gains, not after.

A weak motor *can* still bring the pendulum up, but not along a slow path: it must swing the pendulum back and forth, adding a little energy each time, like pushing a child on a swing. That idea needs the energy of the pendulum, which is exactly where Session 6 starts.

Question: what if the model is wrong?

The controller uses the mass it *believes*, call it $\hat{m}$. The real pendulum has mass m. Take $\hat{m} = 0.5\,\text{kg}$ and $m = 0.6\,\text{kg}$: 20 % heavier. We ask the controller to hold the pendulum horizontal, $\theta_d = \pi/2$, where gravity is strongest.

What goes wrong. The controller supplies $\hat{m} g \ell \sin(e + \theta_d)$ but gravity pulls with $mg\ell \sin(e + \theta_d)$. The cancellation leaves a residue

$$(\hat{m} - m)\, g \ell \sin(e + \theta_d),$$

a torque the controller does not know about: a *disturbance*, exactly as in Session 4.

A constant unknown torque leaves an offset (Session 4). The next slide computes it.

In `pendulum_tracking.py`, mark `[MODEL]`: `M_MODEL` is $\hat{m}$, used in `controller()`; `M_REAL` is m, used in `plant()`. Run `'heavy'`.

The offset left by a wrong mass

Same three steps as the offset of Session 4.

Step 1. At rest nothing moves, and for regulation the target does not move either: $\dot{e} = \ddot{e} = 0$ and $\dot{\theta}_d = \ddot{\theta}_d = 0$. The error dynamics (true mass) then says $0 = -mg\ell \sin(e_{ss} + \theta_d) + u$.

Step 2. The controller at rest gives $u = \hat{m}g\ell \sin(e_{ss} + \theta_d) + \hat{m}\ell^2(-k_p e_{ss})$.

Step 3. Put Step 2 into Step 1 and solve:

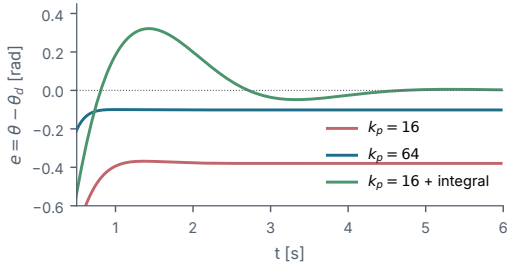
$$\hat{m}\ell^2 k_p \, e_{ss} = (\hat{m} - m)\, g\ell \sin(e_{ss} + \theta_d) \quad \Longrightarrow \quad \boxed{e_{ss} \approx \frac{(\hat{m} - m)\, g \sin\theta_d}{\hat{m}\, \ell\, k_p}}$$

(e_{ss} appears on both sides; for a small offset, $\sin(e_{ss} + \theta_d) \approx \sin\theta_d$.)

What the simulation shows

Holding $\theta_d = \pi/2$ with a pendulum 20 % heavier than the model ($\hat{m} = 0.5$, $m = 0.6$):

- $k_p = 16$: the formula predicts $e_{ss} \approx \frac{(0.5-0.6)\times 9.81}{0.5\times 0.3\times 16} = -0.41$ rad; the simulation gives -0.38 rad, the exact solution of $\hat{m}\ell k_p e_{ss} = (\hat{m} - m)g\sin(e_{ss} + \theta_d)$: the pendulum settles where $\sin(e_{ss} + \theta_d) = 0.93$, not 1. The gap is the approximation, not a numerical error. It hangs about 22° below the horizontal: the controller believes a lighter pendulum and supplies too little torque.
- $k_p = 64$: four times stiffer, offset four times smaller, -0.10 rad.
- $k_p = 16$ with integral action (Session 4): the offset goes to zero.



In `pendulum_tracking.py`, mark `[MODEL]`: run `'heavy'`, `'heavy_high_k'` and `'heavy_integral'`.

Three fixes, and a fourth idea

The offset formula tells us what helps:

$$e_{ss} \approx \frac{(\hat{m} - m)\, g \sin\theta_d}{\hat{m}\, \ell\, k_p}.$$

1. **A better model.** Shrink $\hat{m} - m$: measure the mass. Cheap when possible, impossible when the load changes (a robot picking up different objects).
2. **A stiffer feedback.** Raise k_p: $k_p = 64$ cut the offset from 0.38 to 0.10 rad, but the torque hit the 3 N m limit on the way (`'heavy_high_k'`), and in Session 4 we saw that high gains amplify sensor noise.
3. **Integral action** (Session 4). Add $-k_i z$ to the bracket, with $z = \int e\, dt$: the offset goes to zero (green curve, `'heavy_integral'`).

Each one is a valid engineering answer: a better model removes the uncertainty, and integral action removes the offset without ever knowing the mass. When the parameters are unknown or change during operation, a fourth approach is to *estimate* them online and update the controller as it runs. That is **adaptive control**, Module 3. Whether the estimate $\hat{m}$ also converges to the true m is a separate question: it needs enough *excitation* (Exercise 4).

Tracking with a wrong model. On the swing-up path the 20 % heavier pendulum still reaches the top, with up to 0.37 rad of error on the way (`'heavy_swing'`).

Exercise 4: weighing the pendulum with the offset

A new load of unknown mass m is mounted. The controller still believes $\hat{m} = 0.5\,\text{kg}$, uses $k_p = 16$, and holds $\theta_d = \pi/2$. Once it settles, the encoder reads $e_{ss} = -0.200\,\text{rad}$.

1. Before any calculation: from the sign of e_{ss} alone, is the load heavier or lighter than the controller believes?
2. Use the exact steady-state relation $\hat{m}\ell^2 k_p\, e_{ss} = (\hat{m} - m)\, g\ell \sin(e_{ss} + \theta_d)$ to find m.
3. Could you weigh the pendulum the same way while holding it at the top? What does that say about learning an unknown parameter?

Exercise 4: solution

Step 1. $e_{ss} < 0$: the pendulum hangs below the target, so the controller supplies too little torque against gravity. The load is *heavier* than believed.

Step 2. Solve for m and substitute, with $\sin(\frac{\pi}{2} - 0.2) = 0.980$:

$$m = \hat{m} - \frac{\hat{m}\,\ell\,k_p\,e_{ss}}{g\sin(e_{ss} + \theta_d)} = 0.5 + \frac{0.5 \times 0.3 \times 16 \times 0.2}{9.81 \times 0.980} = 0.550\,\text{kg}.$$

The simulation used 0.55 kg: the load was weighed without a scale. Putting $\hat{m} = 0.55$ in the controller removes the offset. That is one step of adaptive control, done by hand.

Step 3. Not from the offset. At the top $\sin(e_{ss} + \pi) \approx 0$: gravity has no lever arm, the offset is zero *whatever* the mass, and it carries no information about m. The mass could still be identified from a *dynamic* response, for example by accelerating the pendulum, because m also appears in the inertia term.

Lesson. To learn a parameter, the system must be where that parameter matters. In adaptive control (Module 3) this is called **excitation**.

Why we need a new tool: next session

With a perfect model, the error equation was linear, $\ddot{e} + k_d\dot{e} + k_p e = 0$, and eigenvalues told us $e \to 0$. With a wrong mass (friction still known), put the controller in the true error dynamics and divide by $m\ell^2$:

$$\boxed{\ddot{e} + \frac{\hat{m}}{m}\, k_d\, \dot{e} + \frac{\hat{m}}{m}\, k_p\, e = \frac{\hat{m} - m}{m}\left[\frac{g}{\ell}\sin(e + \theta_d) + \ddot{\theta}_d\right]}$$

The right-hand side depends on e through a sine: the error dynamics are **nonlinear again**. Eigenvalues of a linearization can still describe local stability, but they no longer establish stability of the nonlinear system far from the linearization point. Today we settled for simulating it. (At rest, with $\ddot{\theta}_d = 0$, it gives back the offset formula.)

Module 2 is about answering "does $e \to 0$?" for such equations *without* simulating every case. The tool is an **energy** for the error,

$$V = \tfrac{1}{2}\,\dot{e}^2 + \tfrac{1}{2}\,k_p\,e^2,$$

With the perfect model, along $\ddot{e} = -k_d\dot{e} - k_p e$,

$$\dot{V} = \dot{e}\,\ddot{e} + k_p\,e\,\dot{e} = -k_d\,\dot{e}^2 \le 0\,:$$

the energy never increases. That alone does not prove $e \to 0$: $\dot{V}$ is zero whenever $\dot{e} = 0$, even if $e \neq 0$. One more argument (LaSalle's) closes the gap. This is **Lyapunov's method**, next session; it will also tell us how large a model error the controller can tolerate.

Lab 5: running your own scenario

Same procedure as Lab 4. At the end of `SCENARIOS` in `pendulum_tracking.py`, add one line per case, for example

```
    'A1_T2':     dict(path='swing', T=2.0),
    'B1_m055':   dict(path='hold', target=np.pi/2, m=0.55),
```

set `SCENARIO` to that name, and run. Each run prints the maximum error, the final error and the peak torque, saves the plots to `<name>.png` (in Colab, folder icon on the left), and shows the animation.

Key	Meaning	Default	Where on the slides
`path`, `target`	`'hold'`, `'sine'`, `'swing'`, `'ramp'`, `'spin'`; angle to hold	`'swing'`, π	regulation vs tracking
`T`	duration of the swing-up [s]	3	the path
`law`	`'ct'` computed torque or `'pd'` PD + acceleration feedforward	`'ct'`	what happens
`kp`, `kd`	gains of the error equation	16, 8	the controller
`m`	real mass [kg]; the controller always believes 0.5	0.5	wrong model
`ki`	integral gain	0	three fixes
`umax`	motor limit [N·m]	3	the motor limit

Lab 5: what to do

Part A: the path and the torque

1. Run the swing-up with $T = 3,\ 2,\ 1,\ 0.5$ s. Table: T, maximum velocity $\frac{\pi}{2}\,\frac{\pi}{T}$, and the printed peak torque. Does the peak grow? Explain with the four terms of the controller which term is growing.
2. With $T = 3$ s, run `umax` $= 3,\ 1.6,\ 1.5,\ 1.2$. For each limit, say whether the pendulum *reaches* the top, and whether it *tracks* the path accurately (printed maximum error). Compare with the largest torque the path requires, $mg\ell + b\,\dot{\theta}_{d,\max} = 1.55$ N m. Reaching the top and tracking the path are not the same thing.

Part B: the wrong model

3. Hold $\theta_d = \pi/2$ with $m = 0.45,\ 0.5,\ 0.55,\ 0.6$ kg. Table: printed final error next to the formula $e_{ss} = (\hat{m} - m)\,g \sin\theta_d / (\hat{m}\ell k_p)$. Which sign does a *lighter* pendulum give, and why?
4. Repeat $m = 0.6$ with `kp` $= 16,\ 36,\ 64$. Does the offset follow $1/k_p$?
5. Repeat $m = 0.6$, `kp` $= 16$, with `ki` $= 5,\ 20,\ 60$. Which one would you keep? (One or two sentences, with the images.)
6. Hold $\theta_d = \pi$ (the top) with $m = 0.6$. Explain in two sentences why the final error is practically zero here but not at $\pi/2$.

Lab 5 is Homework 5. Every answer is a table, a printed number, an image saved by the program, or one or two sentences. No new code is needed.

Wrap-up and next session

Today

- regulation holds a point, tracking follows $\theta_d(t)$; the error $e = \theta - \theta_d$ is what every controller works on;
- the error obeys $m\ell^2\ddot{e} = -mg\ell\sin(e + \theta_d) - b(\dot{e} + \dot{\theta}_d) - m\ell^2\ddot{\theta}_d + u$;
- computed torque cancels the known terms and adds a PD on the error: $\ddot{e} + k_d\dot{e} + k_p e = 0$ at every angle, no linearization;
- it swings the pendulum up along a slow path, if the motor can supply $mg\ell$;
- a wrong mass leaves an offset $e_{ss} \approx (\hat{m} - m)g\sin\theta_d/(\hat{m}\ell k_p)$.

Next session: Lyapunov

- an energy for the error, $\dot{V} \leq 0$, and LaSalle's argument for $e \to 0$;
- the pendulum's own energy as the first example;
- why this works when eigenvalues do not: wrong model, saturation, nonlinear feedback.

Homework 5: due before the next session

Complete items 1 to 6 of Lab 5 and submit a **single PDF** containing the requested tables, images and brief answers. Upload the PDF to the *Homework 5* assignment on Blackboard by the posted deadline. Submissions by email will not be accepted.